

1. Filter 의 개념과 동작 위치

Filter

공식 문서는 Filter 를 **Servlet API 관점**에서 설명한다.

Servlet API 에서는 `jakarta.servlet.Filter` 를 추가하여, 필터들의 처리 체인과 최종 대상 서블릿의 앞뒤로 **interception-style logic** 을 적용할 수 있다.

여기서 두 가지 표현이 등장한다.

하나는 **처리 과정의 전후 양쪽에 개입한다**는 점이다.

요청이 들어올 때 한 번 로직을 실행하고, 응답이 나갈 때 다시 제어권이 돌아온다.

다른 하나는 하나의 필터가 아니라 **여러 필터가 이어진 체인 구조**라는 점이다.

요청은 필터들을 순서대로 통과한 뒤 대상 서블릿에 도달한다.

```
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) {  
    // 1. 요청이 들어올 때 실행되는 전처리  
    chain.doFilter(req, res);    // 다음 필터 또는 대상 서블릿으로 위임  
    // 2. 응답이 나갈 때 제어권이 돌아와 실행되는 후처리  
}
```

`chain.doFilter()` 호출을 기준으로 전후가 갈리며, 이 호출을 생략하면 요청이 다음 단계로 넘어가지 않고 그 자리에서 끝난다.

동작 위치가 왜 중요한가?

필터는 **Servlet Container 레벨에서 동작하는 서블릿 스펙의 구성 요소**이다. 즉, Spring MVC 의 `DispatcherServlet` 조차도 하나의 서블릿일 뿐이다. 따라서 필터는 `DispatcherServlet` 에 요청이 도달하기 전과 응답이 나간 후에 개입한다.

이 위치 덕분에 필터는 Spring MVC 의 컨트롤러 매핑 여부와 무관하게 **거의 모든 요청에 대해 공통 처리**를 할 수 있다.

ex : 인코딩 설정, 요청/응답 래핑, 헤더 조작, CORS 등

즉, 예시와 같이 웹 요청 자체의 낮은 레벨 처리가 필터에 잘 어울린다.

Spring 에서의 필터 위치

Filter 는 순수 서블릿 스펙 개념이지만, Spring 은 이를 편하게 쓰도록 통합 지점을 제공한다.

공식 문서

- **spring-web** 모듈은 여러 내장 필터 구현체를 제공한다
- 서블릿 필터는 **web.xml** 이나 서블릿 애노테이션으로 설정할 수 있다
- Spring Boot 에서는 필터를 Bean 으로 선언하면 알아서 등록된다

즉, 개념은 서블릿 표준을 따르면서 실제 구성은 Spring / Spring Boot 의 방식으로 자연스럽게 녹아드는 것이다.

정리

Filter

- 서블릿 컨테이너 레벨에서 요청 처리 체인의 앞뒤에 개입하는 **서블릿 표준 구성 요소**이다
- **DispatcherServlet** 보다 **바깥에 위치**하기 때문에, 거의 모든 요청에 대한 공통 처리 혹은 저수준 처리에 적합하다

